



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho 0 - Projeto e Análise de Algoritmos
Gerador de obras de arte aleatórias

Arthur Felipe Campos [6559]

Sumário

1 Introdução	3
2 Organização do projeto	3
3 Implementação	4
3.1 Representação do quadro e constantes	4
3.2 Moldura e inicialização	5
3.3 Geração das figuras básicas	6
3.4 Sorteio das posições e dos tipos	7
4 Compilação e execução	8
4.1 Compilação	8
4.2 Execução	9
5 Resultados	12
6 Parte específica do aluno	14
6.1 Ideia da obra criativa	14
6.2 Resultado da obra criativa	15
7 Decisões de implementação	16
8 Conclusão	17
9 Referências	17

1 Introdução

O trabalho consiste em um programa em linguagem C para montar obras de arte usando caracteres. O quadro tem 20 linhas por 80 colunas, com uma moldura formada por hífens (para a parte superior e inferior) e barras verticais (para as laterais).

O usuário escolhe a figura desejada através do menu interativo, podendo ser um asterisco único, uma figura de “soma” (ou estrela) com asteriscos, uma letra X, uma mistura aleatória das três ou a obra criativa da opção 5. Em seguida, informa quantas figuras quer colocar no quadro. A posição de cada figura é sorteada e desenhada. Ao final, exibimos o quadro no terminal.

2 Organização do projeto

O projeto foi modelado utilizando um TAD, sendo possível, portanto, centralizar de maneira mais eficaz a lógica do programa.

Na Figura 1, é possível observar a pasta **headers** com os arquivos **tad_matriz.h** e **utils.h**. O primeiro contém a declaração da struct principal do tad **MatrizDesenho** e funções de geração dos quadros, já o segundo, funções auxiliares de verificação.

A pasta **sources**, por sua vez, contém os mesmos arquivos, porém com a extensão **.c**, o que significa que correspondem à implementação da lógica dos arquivos de headers.

O **main.c** faz um include em **utils.h**, permitindo a utilização de todas as funções que foram implementadas. Ele é o responsável pelo menu interativo e as chamadas das funções do tad **MatrizDesenho**. Ou seja, é ele que coordena toda a execução.

Por fim, temos o **Makefile**, responsável por facilitar a execução do programa. Ele compila todos os arquivos do código e gera o **programa.exe**, o executável funcional para o usuário.

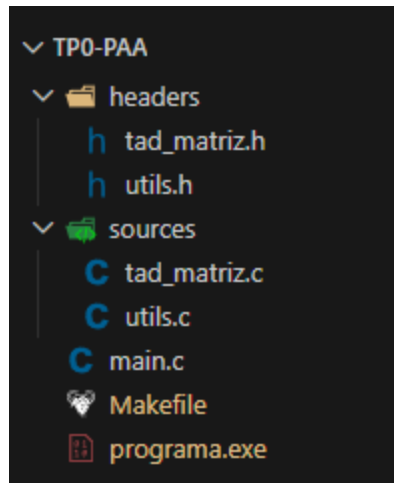


Figura 1 - Repositório

3 Implementação

3.1 Representação do quadro e constantes

A **struct MatrizDesenho** contém apenas uma matriz de 20 linhas por 80 colunas. Os índices da matriz vão de 0 a 19 nas linhas e de 0 a 79 nas colunas.

A Figura 2 mostra uma decisão importante que facilitou a implementação como um todo: o uso de constantes. Basicamente **vazio** representa um espaço 'nulo' na matriz, **linha_i** e **coluna_j** representam o tamanho total da matriz, e **lsup_i** e **lsup_j** representam os limites superiores das linhas e colunas da matriz, ou seja, a área útil, não preenchida pelas bordas.

Utilizar constantes facilita o desenvolvimento, uma vez que é preciso editar os valores em apenas um local, refletindo em todo algoritmo. Um excelente caso de exemplo foi o **vazio**, que inicialmente era um conjunto de **“!!!”**, mas que foi substituído por **‘ ‘**, já que é mais fácil lidar com apenas um caractere do que vários na linguagem C.

```

#define linha_i 20
#define coluna_j 80

#define lsup_i 18
#define lsup_j 78

typedef struct  MatrizDesenho{
    char matriz_desenhos[linha_i][coluna_j];
} MatrizDesenho;

```

Figura 2 - Constantes e TAD

3.2 Moldura e inicialização

A função ***InicializaMatriz*** percorre todas as posições da matriz, usando os índices de linha de 0 a 19 e os índices de coluna de 0 a 79. Ela preenche as linhas superior e inferior com hífens, as colunas laterais com barras verticais e todas as demais posições com espaços vazios. Ao final, a matriz está inicializada e pronta para receber as figuras, conforme mostrado na Figura 3.

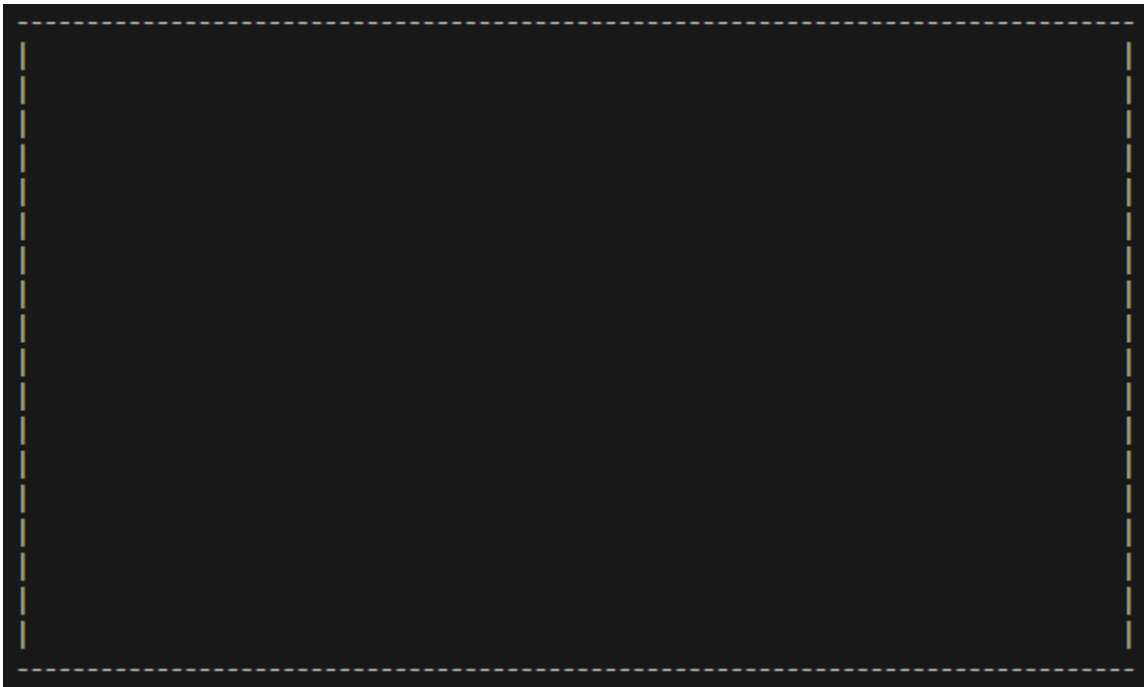


Figura 3 - Moldura do quadro gerado

3.3 Geração das figuras básicas

Há 3 tipos de figuras disponíveis, e para cada uma delas temos uma função específica. Todas elas manipulam a matriz de alguma forma, inserindo os asteriscos nas posições corretas, formando a figura esperada.

Além disso, a função ***GeraFigurasAleatorias*** utiliza as três funções disponíveis, o que promove o reuso de código, vide Figura 4. Nessa função, sorteamos um número de 1 a 3 para escolher qual será o desenho. Depois, o código gera exatamente uma unidade dessa figura. Fazemos isso por n vezes, ou seja, o número de figuras que o usuário digitar.

Opção	Função	Posições escritas
1	GeraAsterisco	Ponto único, “centro”
2	GeraEstrela	Centro, acima, abaixo, esquerda e direita.
3	GeraX	Centro e quatro diagonais.
4	GeraFigurasAleatorias	Uma escolha aleatória entre as três.

```
void GeraFigurasAleatorias(MatrizDesenho *matriz, int num, int min_j, int max_j){
    for (int i=0; i<num; i++){
        int controle_switch = (rand() % 3) + 1;

        switch (controle_switch){

            case 1:
                GeraAsterisco(matriz, 1, min_j, max_j);
                break;

            case 2:
                GeraEstrela(matriz, 1, min_j, max_j);
                break;

            case 3:
                GeraX(matriz, 1, min_j, max_j);
                break;

            default:
                break;
        }
    }
}
```

Figura 4 - Função ***GeraFigurasAleatorias***

3.4 Sorteio das posições e dos tipos

Foi utilizado o ***srand(time(NULL))*** como semente para geração dos números aleatórios. Ele foi inserido no main.c, assim podemos usar em todo código.

main.c

```
srand(time(NULL));
```

Na Figura 5, é possível observar a função ***GeraAsterisco***, que utiliza o ***rand()*** para randomizar os valores de i e j, o que nos permite posicionar o caractere em uma posição aleatória da matriz. Vale ressaltar que o ***rand()*** tem um intervalo de geração, neste caso definidos como ***lsup_i*** para o ***aleatorio_i*** e ***(max_j - min_j + 1)*** para o ***aleatorio_j***. A implementação do intervalo de ***aleatorio_j*** será explicada posteriormente nesse documento, na **seção 6**.

```
void GeraAsterisco(MatrizDesenho *matriz, int num, int min_j, int max_j){
    int max = 150;
    int count_validos = 0;

    while (count_validos < num && max > 0){
        int aleatorio_i = (rand() % lsup_i) + 1;
        int aleatorio_j = (rand() % (max_j - min_j + 1)) + min_j;

        if (matriz -> matriz_desenhos[aleatorio_i][aleatorio_j] == vazio){
            matriz -> matriz_desenhos[aleatorio_i][aleatorio_j] = '*';

            count_validos++;
        }

        max--;
    }
}
```

Figura 5 - Função ***GeraAsterisco***

4 Compilação e execução

4.1 Compilação

O Makefile é crucial para a implementação e execução, pois compila todos os arquivos da pasta **sources** e o **main.c** com apenas uma palavra no terminal. Foi utilizado gcc com **-Wall** e **-Wextra** para ativar avisos. O **-Iheaders** informa ao compilador o caminho que deve buscar os **headers**, os arquivos **.h**.

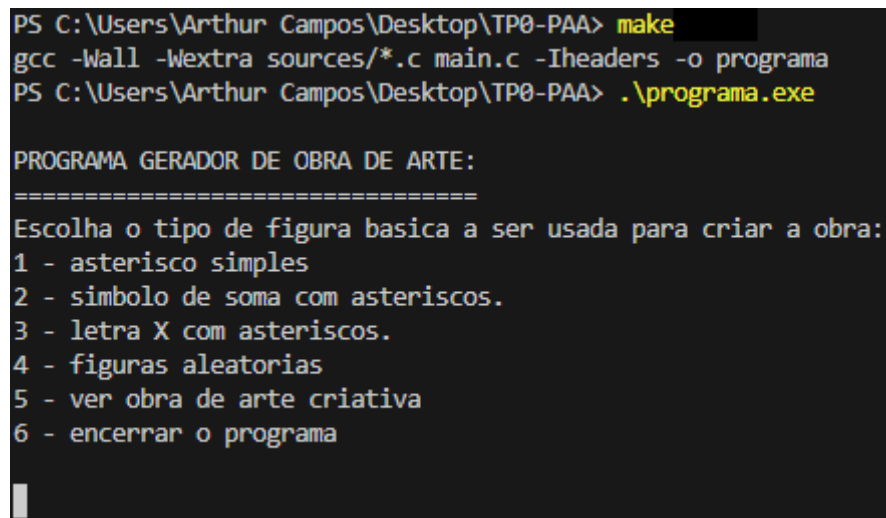
Makefile

```
all:
    gcc -Wall -Wextra sources/*.c main.c -Iheaders -o programa
```

Para compilar e executar o programa, siga os passos abaixo:

1. Navegue até pasta do programa (no **Linux** use: **cd caminho/para/a/pasta/** ; no Windows use: **cd caminho\para\pasta**);
2. Digite e execute o comando **make**;
3. Execute o programa produzido pelo compilador, usando **./programa** em Linux ou **programa.exe** em Windows com gcc.

Na Figura 6, podemos ver o terminal após executar os passos acima. O programa está compilado e pronto para ser utilizado pelo usuário.



```
PS C:\Users\Arthur Campos\Desktop\TP0-PAA> make
gcc -Wall -Wextra sources/*.c main.c -Iheaders -o programa
PS C:\Users\Arthur Campos\Desktop\TP0-PAA> .\programa.exe

PROGRAMA GERADOR DE OBRA DE ARTE:
=====
Escolha o tipo de figura basica a ser usada para criar a obra:
1 - asterisco simples
2 - simbolo de soma com asteriscos.
3 - letra X com asteriscos.
4 - figuras aleatorias
5 - ver obra de arte criativa
6 - encerrar o programa
```

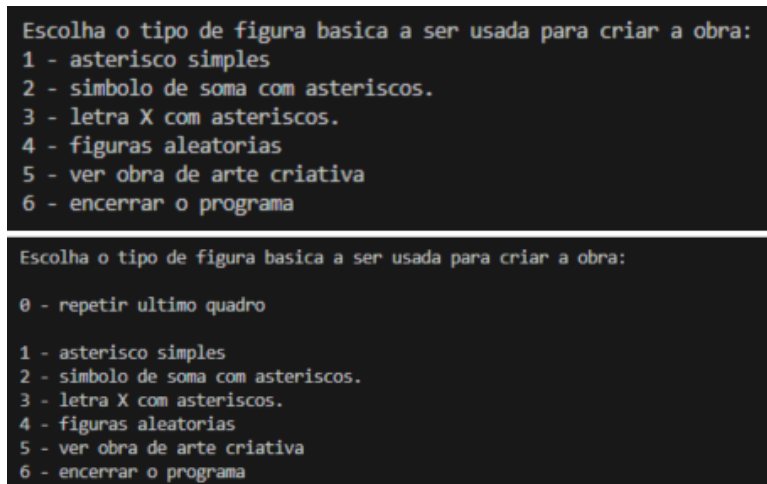
Figura 6 - Compilação e início da execução

4.2 Execução

O sistema permite ao usuário realizar diferentes ações através do menu interativo. Ele funciona de dois modos: o primeiro, quando o usuário inicia o programa pela primeira vez e ainda não gerou obras de arte. O segundo, quando já foi gerado alguma obra de arte, e o usuário quer repetir a mesma, com o exato número de figuras, a fim de pôr em prova a aleatoriedade do sistema.

Na Figura 7 podemos observar as duas possibilidades de menu, com e sem a opção de repetir o último quadro. Ressaltando que a opção 0 aparece **apenas** quando **já foi gerado** pelo menos uma obra de arte.

Além disso, a Figura 8 mostra as verificações realizadas antes de inserir qualquer desenho na matriz. Checamos se o usuário digitou ou não uma opção válida; se ele quer repetir quadros, mas sem ter gerado quadros; se o usuário quer encerrar o programa; e claro, se devemos ou não mostrar o terminal com a opção 0, que repete o último quadro.



```
Escolha o tipo de figura basica a ser usada para criar a obra:
1 - asterisco simples
2 - simbolo de soma com asteriscos.
3 - letra X com asteriscos.
4 - figuras aleatorias
5 - ver obra de arte criativa
6 - encerrar o programa

Escolha o tipo de figura basica a ser usada para criar a obra:
0 - repetir ultimo quadro
1 - asterisco simples
2 - simbolo de soma com asteriscos.
3 - letra X com asteriscos.
4 - figuras aleatorias
5 - ver obra de arte criativa
6 - encerrar o programa
```

Figura 7 - Tipos de menu interativo

```

printf("\nEscolha o tipo de figura basica a ser usada para criar a obra:\n");

if (existe_ultimo_quadro){
    printf("\n0 - repetir ultimo quadro\n\n");
}

printf("1 - asterisco simples\n");
printf("2 - simbolo de soma com asteriscos.\n");
printf("3 - letra X com asteriscos.\n");
printf("4 - figuras aleatorias\n");
printf("5 - ver obra de arte criativa\n");
printf("6 - encerrar o programa\n\n");

scanf("%d", &controle_switch);

if (controle_switch == 6){
    sair = 1; continue;
}

int repetindo = 0;

if (controle_switch == 0){
    if (existe_ultimo_quadro == 0){
        controle_switch = 5;
        printf("\nopcao invalida. ativando obra criativa\n");
    }

    else{
        controle_switch = ultima_opcao;
        num_figuras = ultimo_num_figuras;
        repetindo = 1;
    }
}

if (repetindo == 0){
    if ((controle_switch < 1 || controle_switch > 5)){
        controle_switch = 5;
        printf("\nopcao invalida. ativando obra criativa\n");
    }
}

```

Figura 8 - Verificações feitas no menu

Por fim, a Figura 9 permite visualizar o **switch case**, que recebe a entrada do usuário e realiza a chamada das funções de desenho. É importante ressaltar que todas as entradas do usuário são validadas, inclusive o número de figuras desejadas, nesse caso pela função **VerificaQntdValida**, que será explicada posteriormente na **seção 7**. Além disso, a cada interação do usuário a matriz de desenhos é preenchida com vazios, a fim de não gerar conflitos entre figuras novas e antigas. Ao final, imprimimos a matriz no terminal.

```
printf("\ndigite a quantidade de figuras desejadas:\n");
scanf("%d", &num_figuras);

VerificaQntdValida(&num_figuras);

InicalizaMatriz(&matriz);

switch (controle_switch){
case 1:
    GeraAsterisco(&matriz, num_figuras, 1, lsup_j);
    break;

case 2:
    GeraEstrela(&matriz, num_figuras, 1, lsup_j);
    break;

case 3:
    GeraX(&matriz, num_figuras, 1, lsup_j);
    break;

case 4:
    GeraFigurasAleatorias(&matriz, num_figuras, 1, lsup_j);
    break;

case 5:
    printf("\nas figuras agora serao espelhadas!\n");
    GeraEspelhos(&matriz, num_figuras);
    break;
}

ImprimeMatriz(&matriz);
```

Figura 9 - Switch Case e verificação da quantidade de figuras

5 Resultados

Após compilar o programa e interagir com ele, é possível visualizar as obras de arte. As Figuras 10, 11, 12, 13 e 14 mostram as obras de arte, sendo elas o asterisco, o símbolo de soma/estrela, o X, as figuras aleatórias e a repetição do último quadro, respectivamente. Todas as execuções solicitaram 15 figuras para desenho.



Figura 10 - somente asteriscos



Figura 11 - Somente 'somas'

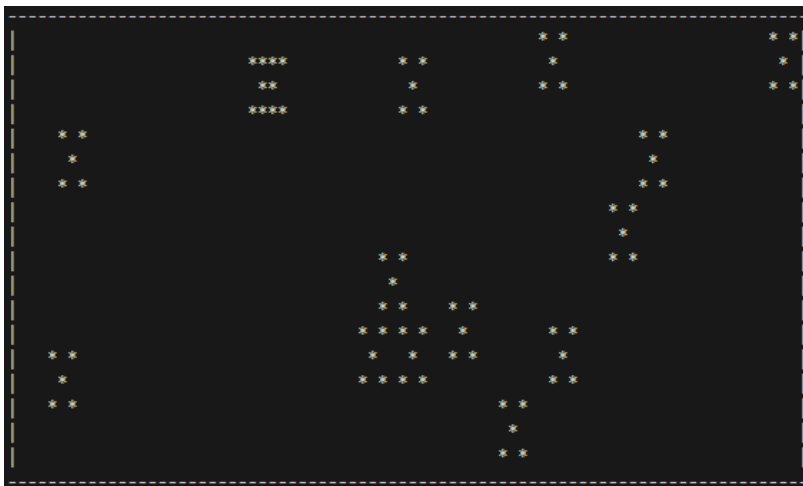


Figura 12 - Somente X

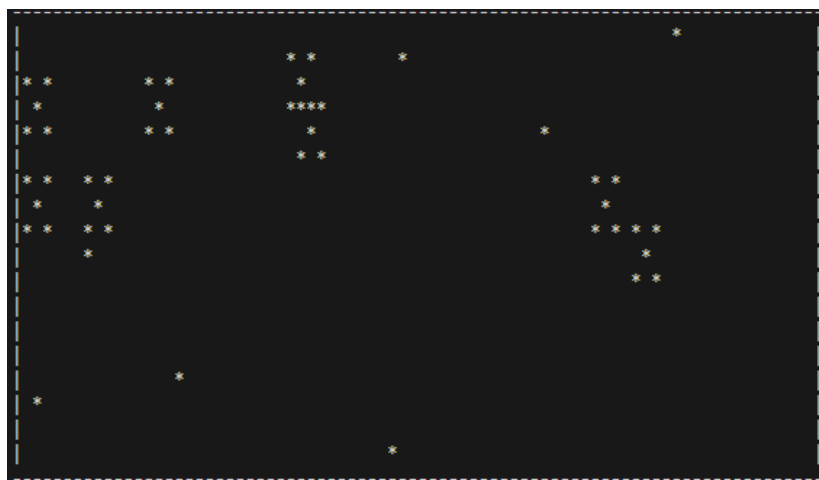


Figura 13 - Figuras aleatórias



Figura 14 - Repetição do último quadro (aleatórias)

6 Parte específica do aluno

6.1 Ideia da obra criativa

A obra criativa é formada por figuras espelhadas. O programa gera uma mistura aleatória das três formas básicas somente na parte esquerda do quadro, e depois, para cada asterisco encontrado, o código calcula a coluna correspondente no lado direito e insere o caractere. Além das figuras, a função coloca duas barras verticais nas colunas centrais. Elas funcionam como um eixo visual para o espelhamento.

Para prosseguir com a aleatoriedade e garantir o reuso de código, as funções de geração de desenhos foram adaptadas: elas recebem parâmetros que definem o intervalo de geração de desenhos. Dessa forma, os parâmetros *min_j* e *max_j* são informados como 1 e 38, respectivamente, o que garante que os desenhos sejam gerados apenas no lado esquerdo e não ultrapassem a borda central definida. A implementação pode ser observada na Figura 15.

```
void GeraEspelhos(MatrizDesenho *matriz, int num){
    MatrizDesenho aux;

    InicializaMatriz(&aux);

    for (int i=1; i<linha_i-1; i++){
        aux.matriz_desenhos[i][39] = '|';
        aux.matriz_desenhos[i][40] = '|';
    }

    GeraFigurasAleatorias(&aux, num, 1, 38);

    for (int i=1; i<linha_i-1; i++){
        for (int j=0; j<39; j++){
            if (aux.matriz_desenhos[i][j] == '*'){
                int j_espelho = coluna_j - 1 - j;

                matriz -> matriz_desenhos[i][j] = '*';
                matriz -> matriz_desenhos[i][j_espelho] = '*';
            }
        }
    }

    for (int i=1; i<linha_i-1; i++){
        matriz -> matriz_desenhos[i][39] = '|';
        matriz -> matriz_desenhos[i][40] = '|';
    }
}
```

Figura 15 - Implementação da **GeraEspelhos**

Observe que dentro da **GeraEspelhos** usamos a função **GeraFigurasAleatorias**, com **min_j** e **max_j** definidos como 1 e 38.

Vale ressaltar que nas chamadas normais, fora do escopo da obra criativa, ambos os parâmetros são “irrelevantes”, uma vez que têm valor 1 e **lsup_j** (78). Ou seja, todo o espaço disponível no quadro. Dessa forma, o **(aleatorio_j = (rand() % (max_j - min_j + 1)) + min_j)** é o mesmo que dizer **(aleatorio_j = (rand() % 78) + 1)**, gerando então números aleatórios entre 1 e 78.

Observe como o **main.c** utiliza as funções em uso convencional na Figura 16.

```
case 1:
    GeraAsterisco(&matriz, num_figuras, 1, lsup_j);
    break;
```

Figura 16 - Chamada normal no **main.c**

6.2 Resultado da obra criativa

Usando 15 como número de figuras, obtivemos o resultado esperado de espelhamento, vide Figura 17.



Figura 17 - Resultado da obra criativa

7 Decisões de implementação

Nessa seção será detalhado algumas decisões que merecem atenção.

Os algoritmos de inserção das figuras na matriz tem um limite máximo de colisões, definido como 150. Dessa forma, o código tenta por 150 vezes inserir uma figura, e se não encontrar espaço disponível, ele interrompe a execução. O ponto positivo é que eliminamos os loops infinitos, mas o negativo é que para um grande número de figuras, o algoritmo talvez não gere todas elas.

Funções como **VerificaVizinhos** e **VerificaQntdValida** foram usadas para facilitar a legibilidade do algoritmo. A primeira, faz a checagem dos vizinhos das figuras X e “soma” (ou estrela). Foi usado uma variável de controle como parâmetro para indicar qual figura terá as posições checadas, sendo 1 para estrela e 2 para X, vide Figura 18.

```
int VerificaVizinhos(MatrizDesenho *matriz, int aleatorio_i, int aleatorio_j, int control){
    if (control==1){ //estrela
        if (matriz -> matriz_desenhos[aleatorio_i][aleatorio_j] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i - 1][aleatorio_j] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i + 1][aleatorio_j] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i][aleatorio_j - 1] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i][aleatorio_j + 1] == vazio){
            return 1;
        }
    }

    if (control==2){ //x
        if (matriz -> matriz_desenhos[aleatorio_i][aleatorio_j] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i - 1][aleatorio_j - 1] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i - 1][aleatorio_j + 1] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i + 1][aleatorio_j - 1] == vazio &&
            matriz -> matriz_desenhos[aleatorio_i + 1][aleatorio_j + 1] == vazio){
            return 1;
        }
    }

    return 0;
}
```

Figura 18 - VerificaVizinhos

Já a função **VerificaQntdValida** foi usada para validar a entrada correspondente ao número de figuras solicitadas. Se for maior que 100, limitamos a 100. Menor ou igual a 0, sorteamos um número aleatório. Se não se encaixar em nenhum dos casos, é retornado o próprio número. É possível visualizar tal implementação na Figura 19.

```

int VerificaQntdValida(int *num){
    if (*num <= 0){
        printf("gerando um numero aleatorio de figuras\n");
        *num = (rand() % 100) + 1;
        printf("%d figuras foram geradas\n", *num);
    }

    else if (*num > 100){
        *num = 100;
    }

    return *num;
}

```

Figura 19 - *VerificaQntdValida*

Além disso, a função ***GeraEspelhos*** utiliza uma matriz auxiliar para organizar a geração da obra. As figuras são sorteadas e inseridas inicialmente apenas no lado esquerdo dessa matriz. Em seguida, o programa percorre as posições ocupadas por asteriscos, calcula suas posições simétricas e grava os mesmos asteriscos na matriz principal, formando o lado direito espelhado.

8 Conclusão

O trabalho cumpriu o objetivo de retomar a programação em C e permitiu aplicar, de forma prática, conceitos como estruturas, funções, matrizes, controle de fluxo e geração de números aleatórios. Além disso, a implementação da obra criativa exigiu decisões próprias sobre a organização do código e sobre o uso da aleatoriedade.

9 Referências

[1] STACKOVERFLOW. Como gerar números aleatórios em C

Disponível em:

<https://pt.stackoverflow.com/questions/210172/gerar-n%C3%BAmeros-aleat%C3%B3rios-em-c>